Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Unit TestsThis test suite validates an AWS Glue PySpark ETL job's transformation logic without performing actual I/O operations. Below is a clear breakdown of each component:---

1. Setup and Initialization

Import and Path Configuration

```
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', '..'))
```

- Adds the parent directory to Python's module search path- Enables importing the main job module from `main.job`

Spark Session Creation

```
def get_spark(): spark = SparkSession.builder \
.appName("test-customer-order-etl") \ .master("local[2]") \
.config("spark.sql.shuffle.partitions", "2") \ .getOrCreate()
```

- Creates a local Spark session with 2 worker threads- Reduces shuffle partitions to 2 for faster testing- Reuses existing session if already created---

2. Individual Transformation Tests

Test 1: Job Parameters Loading

```
def test_get_job_parameters():
```

Purpose: Validates that job configuration parameters are correctly loaded**Checks:**- All required parameter keys exist (input paths, output paths, catalog settings)- S3 paths start with `s3://` protocol- Returns structured configuration dictionary---

Test 2: Column Name Normalization

```
def test_normalize_columns():
```

Purpose: Ensures column names are standardized to lowercase**Test Data:**

```
Input: ['FirstName', 'LastName', 'Age']Output: ['firstname', 'lastname', 'age']
```

Validation: All column names converted to lowercase for consistency---

Test 3: NULL Value Removal

```
def test_remove_nulls():
```

Purpose: Removes records with NULL values or literal 'Null' strings**Test Data:**

```
Input: 5 records (includes NULL custid, NULL email, 'Null' region)Output: 2 valid
records (custid='1' and custid='4')
```

Logic: Filters out any row containing NULL or the string 'Null' in any column---

Test 4: Duplicate Record Removal

```
def test_remove_duplicates():
```

Purpose: Eliminates duplicate rows from dataset**Test Data:**

```
Input: 5 records (2 duplicates of custid='1', 2 duplicates of custid='2')Output: 3
unique records
```

Validation: Uses DataFrame's `dropDuplicates()` to keep only unique rows---

Test 5: SCD Type 2 Column Addition

```
def test_add_scd_columns():
```

Purpose: Adds Slowly Changing Dimension Type 2 tracking columns**Added Columns:**- `IsActive`: Boolean flag (set to `True` for new records)- `StartDate`: Timestamp when record became active- `EndDate`: Timestamp when record became inactive (initially `None`)- `OpTs`: Operation timestamp for change tracking**Validation:** Verifies all SCD columns exist with correct initial values---

3. I/O Operation Tests (Mocked)

Test 6: S3 Access Validation

```
@patch('main.job.boto3.client')def test_validate_s3_access(mock_boto_client):
```

Purpose: Tests S3 bucket accessibility without actual AWS calls**Mocking Strategy:**- Patches `boto3.client` to return a mock S3 client- Mock returns empty list for `list_objects_v2` call- Verifies function returns `True` and calls S3 API correctly---

Test 7: CSV Data Reading

```
def test_data_reader_read_csv():
```

Purpose: Tests CSV file reading logic without actual file I/O**Mocking Strategy:**- Creates in-memory DataFrame with sample customer data- Patches `DataReader._read_csv` to return mock DataFrame- Validates returned DataFrame structure and record count---

Test 8: Parquet Data Writing

```
def test_data_writer_write_parquet():
```

Purpose: Tests Parquet format writing without actual S3 writes**Mocking Strategy:**- Patches `DataWriter._write_parquet` to skip actual write- Verifies write operation returns success status- Validates correct parameters passed to writer---

Test 9: Hudi Data Writing

```
def test_data_writer_write_hudi():
```

Purpose: Tests Apache Hudi format writing with SCD columns**Hudi Configuration:**

```
{ 'hoodie.table.name': 'ordersummary', 'hoodie.datasource.write.recordkey.field':
'custid', 'hoodie.datasource.write.precombine.field': 'OpTs'}
```

Validation: Ensures Hudi-specific options are correctly applied---

4. End-to-End Pipeline Test

Test 10: Complete ETL Pipeline

```
def test_complete_etl_pipeline():
```

Purpose: Validates entire transformation workflow with realistic data**Pipeline Steps:**1. **Customer Data Processing:** - Input: 5 records (includes duplicates, NULLs, 'Null' strings) - Normalize → Remove NULLs → Remove Duplicates - Output: 1 valid record (custid='1')2. **Order Data Processing:** - Input: 4 records (includes duplicates, NULL quantity) - Normalize → Remove NULLs → Remove Duplicates - Output: 1 valid record (orderid='101')3. **Data Integration:** - Inner join on `custid` - Result: 1 joined record4. **SCD Enhancement:** - Add tracking columns - Verify final structure includes all SCD fields**Final Validation:**- Confirms custid='1' and orderid='101' in result- Verifies `IsActive=True` for new record---

Test 11: Catalog Table Registration

```
def test_register_catalog_table():
```

Purpose: Tests Glue Data Catalog table registration**Mocking Strategy:**- Patches `saveAsTable` to avoid actual catalog writes- Patches `spark.sql` to skip DDL execution- Validates function completes without errors---

5. Test Execution Framework

Test Runner

```
def run_all_tests():
```

Purpose: Executes all tests in sequence with formatted output**Output Format:**

```
============================================================Running AWS Glue PySpark ETL Job Tests============================================================✓ test_get_job_parameters passed✓ test_normalize_columns passed...✓ test_register_catalog_table passed============================================================All tests passed successfully! ✓============================================================
```

Key Testing Principles Applied

1. **Isolation:** Each test is independent and doesn't rely on external resources2. **Mocking:** I/O operations are mocked to avoid AWS dependencies3. **Clarity:** Each test has a single, clear purpose4. **Validation:** Assertions verify both data correctness and transformation logic5. **Traceability:** Tests map directly to production code functionsThis test suite ensures the ETL job's transformation logic is correct before deployment to AWS Glue.